

COMPSCI 361 Assignment 3

Quick Start and Team Task Guide

1 How to Run the Project

1.1 1. Clone the Repository

Clone the project and move into the repository folder:

```
git clone https://github.com/HienMinh58/Assignment-3-COMPSCI-361.git
cd Assignment-3-COMPSCI-361
```

1.2 2. Start the Project with Docker

Run:

```
docker compose up --build
```

This starts the local FastAPI service in Docker.

1.3 3. Add the Local Datasets

Place the assignment datasets in:

```
data/train.csv
data/test.csv
```

These files are not committed to GitHub, so each teammate needs to add them locally.

1.4 4. Run the Local Classifier Metrics Script

To test the current classifier implementations, run:

```
python scripts/run_classifier_metrics.py
```

The script evaluates all classifier flows. Implemented classifiers return metrics, while unfinished classifiers return a `not_implemented` status.

1.5 5. Open the Swagger UI

After starting Docker, open the interactive API documentation at:

<http://localhost:8000/docs>

Useful endpoints include:

- GET `/metrics/nb`
- GET `/metrics/knn`
- GET `/metrics/svm`
- GET `/metrics/ann`

2 Assignment Tasks to Implement

2.1 Task 1: Exploratory Data Analytics

2.1.1 Main File

Implement reusable Task 1 logic in `src/eda.py`.

The final plots, tables, and explanations should later be inserted into the notebook.

2.1.2 Required Work

Task 1 requires:

- Load the training and testing datasets.
- Construct feature vectors for articles.
- Report:
 - number of training articles,
 - number of testing articles,
 - number of extracted features in train,
 - number of extracted features in test.
- Show 5 example articles with their extracted features using a dataframe.
- Create three plots:
 - top-50 term frequency distribution,
 - term frequency distribution per class,
 - class distribution for train and test datasets.

2.2 Task 2: Classification Models Learning

2.2.1 Files to Work On

Each model should be implemented in its own classifier file:

- Naive Bayes: `src/classifiers/nb_classifier.py`
- kNN: `src/classifiers/knn_classifier.py`
- SVM: `src/classifiers/svm_classifier.py`
- ANN: `src/classifiers/ann_classifier.py`

All classifier implementations should expose a consistent interface with the following three core methods:

<code>fit(train_texts, train_labels)</code>	Train the classifier.
<code>predict(test_texts)</code>	Generate predictions for input texts.
<code>evaluate(test_texts, test_labels)</code>	Compute evaluation metrics.

2.2.2 Required Work

Naive Bayes The Naive Bayes reference implementation is already available. The Task 2(a) assignment outputs still need to include:

- top 20 most predictive words for each class,
- top 20 most discriminative words between the two classes,
- a short explanation of which word list describes the classes better.

kNN Implement the kNN classifier in:

`src/classifiers/knn_classifier.py`

Required assignment outputs:

- train a kNN classifier,
- choose k and a distance metric,
- plot a decision boundary,
- explain the impact of k and the distance metric.

SVM Implement the SVM classifier in:

`src/classifiers/svm_classifier.py`

Required assignment outputs:

- train a soft-margin linear SVM,
- train a hard-margin RBF-kernel SVM,
- plot both decision boundaries,
- explain the impact of C and σ .

ANN Implement the ANN classifier in:

`src/classifiers/ann_classifier.py`

Required assignment outputs:

- use one hidden layer,
- test hidden units $h \in \{2, 5, 20, 40\}$,
- use learning rate 0.01,
- run for 100 epochs,
- plot binary cross-entropy loss against hidden units,
- explain the effect of the number of hidden units.